

INFORME DE LA PRÁCTICA EVALUABLE: CONTADOR DE CUENTA ARBITRARIA DE 4 BITS

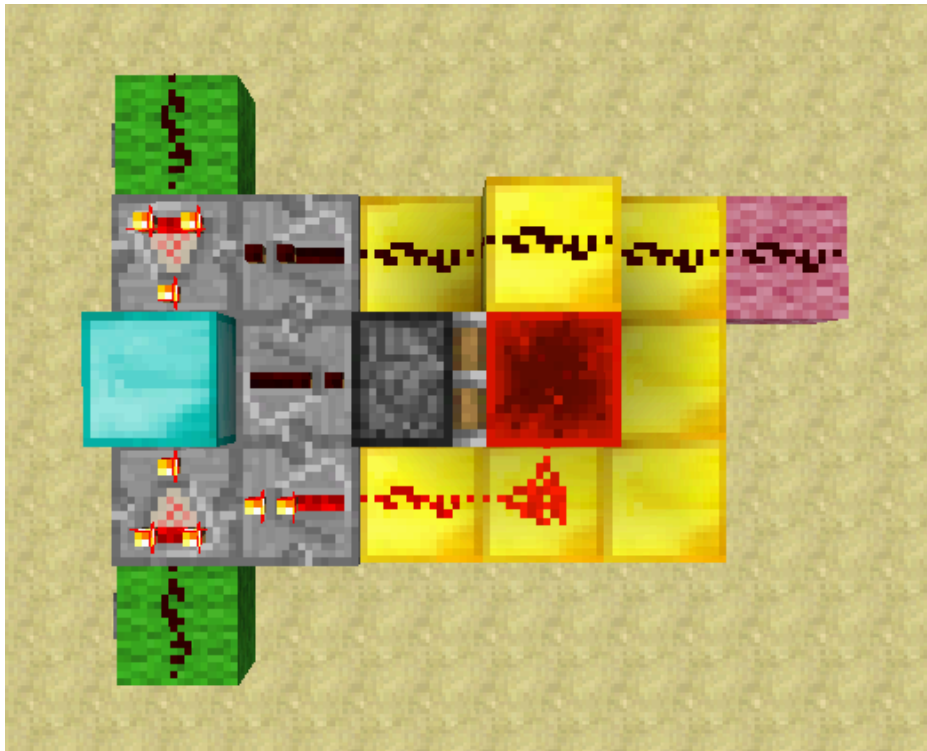


Imagen de la portada: un módulo JK flip flop en Minecraft [1].

Blanco de la Iglesia, David

Computadores I

Rodríguez Fernández, Nicolás

(101104)

1. Índice interactivo

1. Índice interactivo.....	2
2. Objetivo y aspectos relevantes.....	2
3. Diagrama de estados.....	3
4. Tabla de transiciones.....	4
5. Mapas de Karnaugh:.....	6
6. Esquemas de los módulos desarrollados indicando la correspondencia de variables....	7
7. Código en Verilog.....	8
8. Ejemplos de pruebas (Con y sin GTKWave).....	14
9. Conclusiones, referencias y bibliografía.....	20

2. Objetivo y aspectos relevantes

El objetivo de esta práctica es diseñar y programar con Verilog un contador arbitrario de 4 bits. El diseño se explica detalladamente en los apartados desde el 2 hasta el 6, y la programación y resultados en Verilog se detalla tanto en los apartados 7 y 8 como en el código fuente. Este queda adjunto en el archivo .zip llamado «*Practica_entregable*», llamándose el código «*Codigo_Practica_entregable.v*», el diagrama de señales «*contador_arbitrario.dmp*» y el informe «*INFORME PRÁCTICA COMPUTADORES I.pdf*».

Las herramientas usadas durante todo el proceso de la práctica fueron: la máquina virtual de Raúl Alves en Oracle Virtualbox, Verilog, GTKWave, Online Visual Paradigm, Documentos de Google y Visual Studio Code.

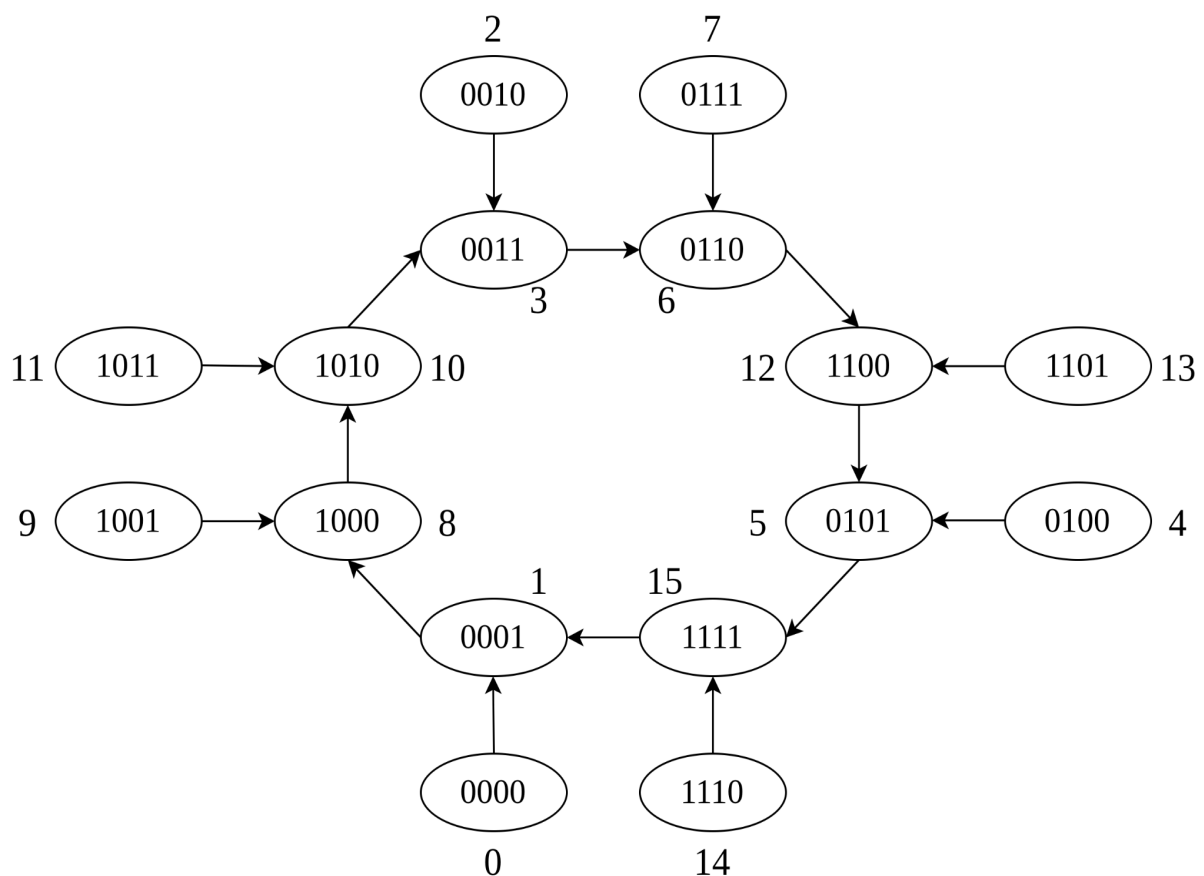
La defensa presencial de esta práctica será realizada por sus dos autores ante el profesor de prácticas correspondiente, el día lunes, 1 de diciembre de 2025 a las 12:00 horas.

3. Diagrama de estados

La secuencia arbitraria dada por el profesor es la siguiente:

« 6 → 12 → 5 → 15 → 1 → 8 → 10 → 3 → *Vuelve a empezar* »

Los estados no seleccionados (los números 0, 2, 4, 7, 9, 11, 13 y 14) irán hacia los estados que más se le parezcan en binario, dando prioridad a los cambios de bit en los que el bit es menos significativo. Por ejemplo, el estado 13 transiciona al 12 de la forma que en binario pasa del 1101 al 1100. El bit menos significativo, el Q_0 , pasa de estar ON [1] a OFF [0]. Realizando este proceso mecánico para cada uno de los estados seleccionados, así pues, nuestro diagrama de estados queda de la siguiente manera:



4. Tabla de transiciones

Tras analizar las transiciones a mano y posteriormente comprobarlas con el siguiente script de python:

```
transiciones = {
    "0000": "0001",
    "0001": "1000",
    "0010": "0011",
    "0011": "0110",
    "0100": "0101",
    "0101": "1111",
    "0110": "1100",
    "0111": "0110",
    "1000": "1010",
    "1001": "1000",
    "1010": "0011",
    "1011": "1010",
    "1100": "0101",
    "1101": "1100",
    "1110": "1111",
    "1111": "0001"
}

cambio = {
    "00": "0  x",
    "01": "1  x",
    "10": "x  1",
    "11": "x  0"
}

titulo = "|  Transiciones  |  J3 K3 |  J2 K2 |  J1 K1 |  J0 K0 |"
print("\r" + "-"*len("  Transiciones  ") + ("T" + "-"*len("  J0 K0  "))* 4 +
"\r")
print(titulo)
print("\t" + "-"*len("  Transiciones  ") + ("T" + "-"*len("  J0 K0  "))* 4 +
"\t")

for (i, j) in transiciones.items():
    print("\t", end="")
    print(i + " -> " + j, end=" ")
    print("\t", end="")
    for c in range(4):
        print(cambio[i[c] + j[c]], end=" | ")
    print()
print("\r" + "-"*len("  Transiciones  ") + ("T" + "-"*len("  J0 K0  "))* 4 +
"\r")
```

Que da como resultado la siguiente salida en pantalla:

Transiciones	J3 K3	J2 K2	J1 K1	J0 K0
0000 -> 0001	0 x	0 x	0 x	1 x
0001 -> 1000	1 x	0 x	0 x	x 1
0010 -> 0011	0 x	0 x	x 0	1 x
0011 -> 0110	0 x	1 x	x 0	x 1
0100 -> 0101	0 x	x 0	0 x	1 x
0101 -> 1111	1 x	x 0	1 x	x 0
0110 -> 1100	1 x	x 0	x 1	0 x
0111 -> 0110	0 x	x 0	x 0	x 1
1000 -> 1010	x 0	0 x	1 x	0 x
1001 -> 1000	x 0	0 x	0 x	x 1
1010 -> 0011	x 1	0 x	x 0	1 x
1011 -> 1010	x 0	0 x	x 0	x 1
1100 -> 0101	x 1	x 0	0 x	1 x
1101 -> 1100	x 0	x 0	0 x	x 1
1110 -> 1111	x 0	x 0	x 0	1 x
1111 -> 0001	x 1	x 1	x 1	x 0

Procedemos a introducir en la siguiente tabla los datos obtenidos:

Transiciones		J ₃	K ₃	J ₂	K ₂	J ₁	K ₁	J ₀	K ₀
0 → 1	0000 → 0001	0	x	0	x	0	x	1	x
1 → 8	0001 → 1000	1	x	0	x	0	x	x	1
2 → 3	0010 → 0011	0	x	0	x	x	0	1	x
3 → 6	0011 → 0110	0	x	1	x	x	0	x	1
4 → 5	0100 → 0101	0	x	x	0	0	x	1	x
5 → 15	0101 → 1111	1	x	x	0	1	x	x	0
6 → 12	0110 → 1100	1	x	x	0	x	1	0	x
7 → 6	0111 → 0110	0	x	x	0	x	0	x	1
8 → 10	1000 → 1010	x	0	0	x	1	x	0	x
9 → 8	1001 → 1000	x	0	0	x	0	x	x	1
10 → 3	1010 → 0011	x	1	0	x	x	0	1	x
11 → 10	1011 → 1010	x	0	0	x	x	0	x	1
12 → 5	1100 → 0101	x	1	x	0	0	x	1	x
13 → 12	1101 → 1100	x	0	x	0	0	x	x	1
14 → 15	1110 → 1111	x	0	x	0	x	0	1	x
15 → 1	1111 → 0001	x	1	x	1	x	1	x	0

En esta tabla marcamos con color rojo todos los estados inválidos y con verde todos los estados válidos. Nótese como los estados inválidos deberían desembocar en estados válidos.

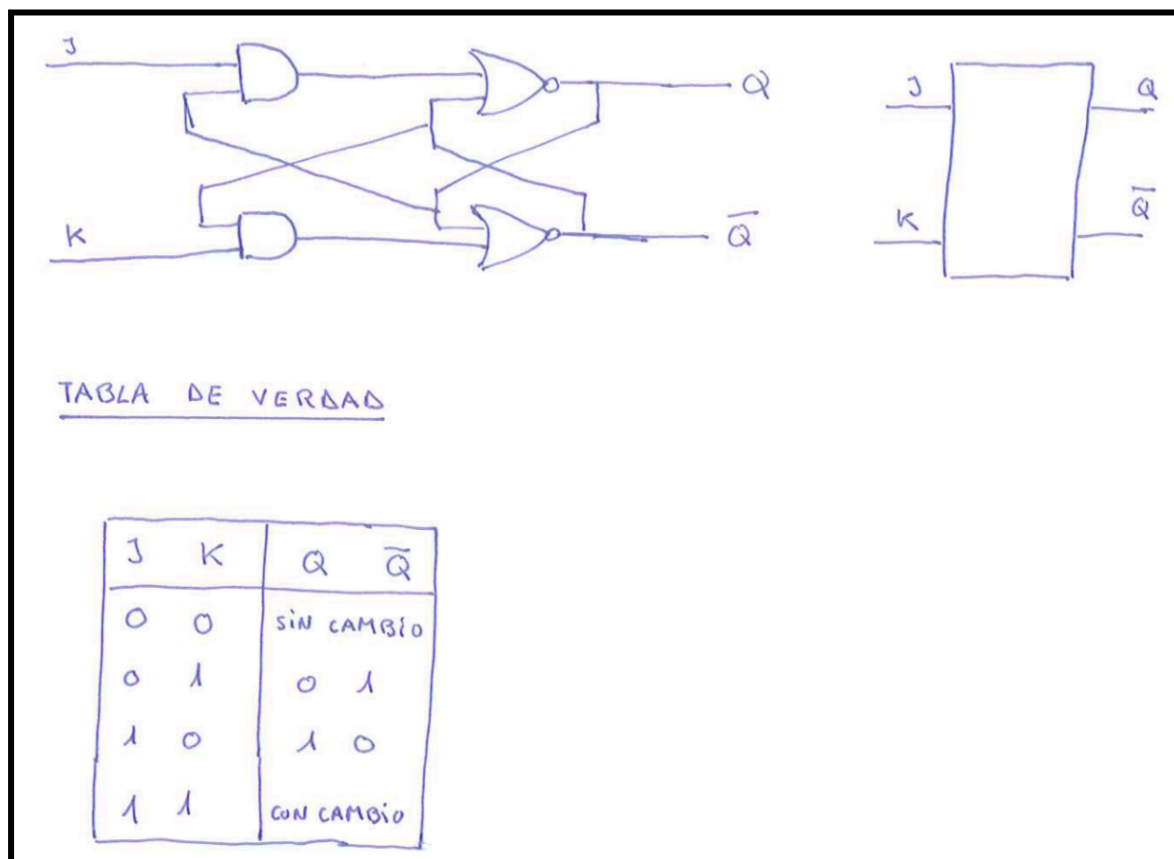
5. Mapas de Karnaugh:

Procedemos ahora a elaborar los mapas de Karnaugh, a partir de los datos de la tabla de transiciones mostrada en el punto anterior.

$Q_1Q_0 \rightarrow$ $Q_3Q_2 \downarrow$	00 (J)	01 (J)	11 (J)	10 (J)		00 (K)	01 (K)	11 (K)	10 (K)	
00	0	1	0	0		x	x	x	x	FlipFlop 1
01	0	1	1	0		x	x	x	x	
11	x	x	x	x		1	0	1	0	
10	x	x	x	x		0	0	0	1	
Ecuación: $J_3 = \overline{Q_1} \cdot Q_0 + Q_2 \cdot Q_1 \cdot \overline{Q_0}$						Ecuación: $K_3 = Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0} + Q_2 \cdot Q_1 \cdot Q_0 + \overline{Q_2} \cdot Q_1 \cdot \overline{Q_0}$				
$Q_1Q_0 \rightarrow$ $Q_3Q_2 \downarrow$	00 (J)	01 (J)	11 (J)	10 (J)		00 (K)	01 (K)	11 (K)	10 (K)	
00	0	0	1	0		x	x	x	x	FlipFlop 2
01	x	x	x	x		0	0	0	0	
11	x	x	x	x		0	0	1	0	
10	0	0	0	0		x	x	x	x	
Ecuación: $J_2 = \overline{Q_3} \cdot Q_1 \cdot Q_0$						Ecuación: $K_2 = Q_3 \cdot Q_1 \cdot Q_0$				
$Q_1Q_0 \rightarrow$ $Q_3Q_2 \downarrow$	00 (J)	01 (J)	11 (J)	10 (J)		00 (K)	01 (K)	11 (K)	10 (K)	
00	0	0	x	x		x	x	0	0	FlipFlop 3
01	0	1	x	x		x	x	0	1	
11	0	0	x	x		x	x	1	0	
10	1	0	x	x		x	x	0	0	
Ecuación: $J_1 = \overline{Q_3} \cdot Q_2 \cdot Q_0 + Q_3 \cdot \overline{Q_2} \cdot \overline{Q_0}$						Ecuación: $K_1 = Q_3 \cdot Q_2 \cdot Q_0 + \overline{Q_3} \cdot Q_2 \cdot \overline{Q_0}$				
$Q_1Q_0 \rightarrow$ $Q_3Q_2 \downarrow$	00 (J)	01 (J)	11 (J)	10 (J)		00 (K)	01 (K)	11 (K)	10 (K)	
00	1	x	x	1		x	1	1	x	FlipFlop 4
01	1	x	x	0		x	0	1	x	
11	1	x	x	1		x	1	0	x	
10	0	x	x	1		x	1	1	x	
Ecuación: $J_0 = \overline{Q_3} \cdot \overline{Q_2} + Q_3 \cdot Q_1 + Q_2 \cdot \overline{Q_1}$						Ecuación: $K_0 = \overline{Q_2} + Q_3 \oplus Q_1$				

6. Esquemas de los módulos desarrollados indicando la correspondencia de variables

Los módulos secuenciales que se van a utilizar en esta práctica son los biestables J-K. A continuación, se detalla el cableado, la simplificación y la tabla de verdad de un biestable J-K (dibujo de elaboración propia):



Sin embargo, a la hora de programarlo con verilog no usaremos este diseño con puertas lógicas, sino que utilizaremos algunas herramientas de esta tecnología que permiten crear un dispositivo con la misma función pero de manera más simplificada.

Creado con el software Visual Paradigm, el esquema de todo el circuito es el siguiente:

<https://drive.google.com/file/d/1IjE0hvrdrRHuglIIhFORudSoOJvID1idy/view?usp=sharing>

[Este enlace debería permitir acceder al esquema, guardado en .svg para obtener la mejor resolución posible. En caso de no poder abrirlo, contactar con David (davidblancoigl@usal.es) o con Nicolás (nicolasrf@usal.es)].

7. Código en Verilog

Vamos a proceder a crear el contador utilizando verilog. En primer lugar, es obvio que necesitaremos usar 4 biestables JK, tal y como indicamos en el esquema del circuito (punto 6). Representaremos los biestables como módulos de verilog que tomarán como entradas J, K y CLK, y como salidas Q y nQ (Q negado o complementado). En primer lugar, y para forzar que el valor de nQ sea el de Q complementado creamos una puerta NOT entre estas salidas.

Utilizaremos ahora algunas herramientas propias de verilog para simplificar la creación del módulo. En primer lugar, forzamos que Q valga cero por defecto usando un bloque initial y, en segundo lugar, creamos un bloque always activado por flanco de subida (posedge de CLK) que controla, mediante un bloque case, el funcionamiento del biestable JK (véase la tabla de verdad de este biestable en el punto anterior).

```
module JK(output reg Q, output wire NQ, input wire J, input wire K, input wire
CLK);
not(NQ,Q);

// Ponemos una condición inicial Q = 0.
initial
    begin
        Q=0;
    end

// Flanco de subida
always @(posedge CLK)
    case ({J,K})
        2'b10: Q=1;
        2'b01: Q=0;
        2'b11: Q=~Q;
    endcase
endmodule
```

Ahora ya podemos crear el contador. Declararemos este contador como un módulo de verilog que tomará como entrada un wire de 4 bits (al que llamaremos Q), que contendrá los valores de las cuatro salidas (Q3, Q2, Q1, Q0). Además, este dispositivo también tomará como entrada el reloj CLK (que dirigirá los cambios de estado en el contador (recordemos, por flanco de subida)).

Para facilitar el trabajo creamos otro wire de 4 bits que contendrá los valores de nQ (salidas negadas).

Entonces, declaramos todos los wires que usará el contador (los mismos que los especificados en el esquema del punto seis) para ahora comenzar a crear las puertas lógicas conectando estos cables, puertas lógicas que terminarán conectándose con los cuatro biestables JK que declaramos al final del módulo. Con esto, el contador estaría completo.

```
module Contador (output wire[3:0] Q, input wire CLK);

// Wire de salidas nQ (Q negada)
    wire [3:0] nQ;

// Wires que van directos a las entradas J y K de los respectivos biestables.
    wire wJ3, wK3, wJ2, wK2, wJ1, wK1, wJ0, wK0;

// Wires que conectan puertas lógicas entre sí
    wire j3_t1, j3_t2;
    wire k3_t1, k3_t2, k3_t3;
    wire j1_t1, j1_t2;
    wire k1_t1, k1_t2;
    wire j0_t1, j0_t2, j0_t3;
    wire k0_t1;

// Cableado de los J y los K
// J3 = nQ1Q0 + Q2Q1nQ0
    and g_j3_1 (j3_t1, nQ[1], Q[0]);
    and g_j3_2 (j3_t2, Q[2], Q[1], nQ[0]);
    or g_J3 (wJ3, j3_t1, j3_t2);
// K3 = Q2nQ1nQ0 + Q2Q1Q0 + nQ2Q1nQ0
    and g_k3_1 (k3_t1, Q[2], nQ[1], nQ[0]);
    and g_k3_2 (k3_t2, Q[2], Q[1], Q[0]);
    and g_k3_3 (k3_t3, nQ[2], Q[1], nQ[0]);
    or g_K3 (wK3, k3_t1, k3_t2, k3_t3);

// J2 = nQ3Q1Q0
    and g_J2 (wJ2, nQ[3], Q[1], Q[0]);
// K2 = Q3Q1Q0
    and g_K2 (wK2, Q[3], Q[1], Q[0]);

// J1 = nQ3Q2Q0 + Q3nQ2nQ0
    and g_j1_1 (j1_t1, nQ[3], Q[2], Q[0]);
    and g_j1_2 (j1_t2, Q[3], nQ[2], nQ[0]);
    or g_J1 (wJ1, j1_t1, j1_t2);
// K1 = Q3Q2Q0 + nQ3Q2nQ0
    and g_k1_1 (k1_t1, Q[3], Q[2], Q[0]);
    and g_k1_2 (k1_t2, nQ[3], Q[2], nQ[0]);
    or g_K1 (wK1, k1_t1, k1_t2);

// J0 = nQ3nQ2 + Q3Q1 + Q2nQ1
    and g_j0_1 (j0_t1, nQ[3], nQ[2]);
    and g_j0_2 (j0_t2, Q[3], Q[1]);
    and g_j0_3 (j0_t3, Q[2], nQ[1]);
    or g_J0 (wJ0, j0_t1, j0_t2, j0_t3);
// K0 = nQ2 + Q3 ⊕ Q1
    xor g_k0_1 (k0_t1, Q[3], Q[1]);
    or g_K0 (wK0, nQ[2], k0_t1);

// Conectamos los wires a las entradas J y K de los biestables
    JK jk3 (Q[3], nQ[3], wJ3, wK3, CLK);
    JK jk2 (Q[2], nQ[2], wJ2, wK2, CLK);
    JK jk1 (Q[1], nQ[1], wJ1, wK1, CLK);
    JK jk0 (Q[0], nQ[0], wJ0, wK0, CLK);
endmodule
```

Finalmente, podemos comenzar a crear el módulo test, que utilizaremos para probar el correcto funcionamiento del contador. Tras declarar Q (wire de 4 bits), CLK como registro (que inicializamos como 0 en el bloque inicial de este módulo) y counter como instancia de Contador (que acabamos de programar), podemos proceder con las pruebas.

En primer lugar, y mediante un bloque always, vamos a hacer que el registro CLK oscile entre 0 y 1. Así, cuando pase de 0 a 1 (flanco de subida), debería activar el contador, cambiando el estado de este.

Ahora pasamos a crear un bloque initial, el cual contendrá el código encargado del display de las pruebas. Sin embargo, y antes de proceder con ello, utilizaremos la instancia de la clase Contador counter para cambiar sus atributos Q. De este modo, podemos forzar que el contador comience en 6, y no en 0 como habíamos declarado por defecto cuando definimos el módulo JK.

Ahora ya podemos comenzar con las pruebas.

Comenzaremos usando las funciones dumpfile (para declarar el nombre del archivo) y dumpvars (para indicar qué variables vamos a analizar) con el objetivo de crear un archivo .dmp que muestre el diagrama de reloj y el comportamiento de los biestables para posteriormente poder abrirlo y estudiarlo con el software GTKwave.

Sin embargo, también hemos incluido pruebas utilizando la función monitor, que imprimirá el valor de las variables indicadas (fundamentalmente las salidas del contador) cada vez que haya un cambio en el valor de CLK (anotado con un delay de 10 unidades de tiempo por cambio).

Por último, llamamos a finish tras 1000 unidades de tiempo para así evitar que el código se ejecute indefinidamente, es decir, este terminará cuando hayan pasado 1000 unidades de tiempo (cuando clock haya oscilado 100 veces).

El código del módulo *module test* quedaría de la siguiente manera:

```
module test;
    reg C; // CLOCK
    wire [3:0] Q; // Salidas
    Contador counter (Q,C); // Instancia del contador

    // El clock oscila cada 10 unidades de tiempo.
    always begin
        #10 C=~C;
    end

    initial begin

        // Estados inválidos:
        // 0 2 4 7 9 11 13 14
        // 0000 0010 0100 0111 1001 1011 1101 1110
        // Ponemos estos números binarios en estas líneas para comprobar cómo desembocan en
        // estados válidos.

        // Ahora, empezamos por el estado 6 (0110 en binario):
        counter.jk3.Q = 'b0;
        counter.jk2.Q = 'b1;
        counter.jk1.Q = 'b1;
        counter.jk0.Q = 'b0;

        // Archivo para GTKWave (opcional si solo quieres ver consola)
        $dumpfile("contador_arbitrario.dmp");
        $dumpvars(2, counter, Q);
        C=0;

        // Monitoreamos la señal de reloj para observar los cambios en la salida.
        // Imprimimos la salida en binario para ver el correcto funcionamiento del contador.
        $monitor($time, ": Clock: %b | Salida Q: %d (Binario: %b)", C, Q, Q);
        #1000 $finish; // terminamos el programa a las 1000 unidades de tiempo para
        // evitar entrar en un bucle infinito.
    end
endmodule
```

El resultado de unir estos tres módulos es un programa que permite usar y también validar el correcto funcionamiento del contador arbitrario pedido. El código es ejecutable a través del archivo «Codigo_Practica_entregable.v» y por la consola terminal de Linux con los siguientes comandos (dentro de la carpeta donde se encuentre el archivo):

```
iverilog Codigo_Practica_entregable.v -o contador
./contador
```

[El comando de ejecución puede ser tanto «contador» como cualquier otra palabra, así que este texto resaltado en morado grisáceo es un ejemplo de cómo se podría ejecutar].

El código completo en verilog quedaría de la siguiente manera:

```
module JK(output reg Q, output wire NQ, input wire J, input wire K, input wire
CLK);
not(NQ,Q);

// Ponemos una condición inicial Q = 0.
initial
begin
    Q=0;
end

// Flanco de subida
always @(posedge CLK)
begin
    case ({J,K})
        2'b10: Q=1;
        2'b01: Q=0;
        2'b11: Q=~Q;
    endcase
endmodule

module Contador (output wire[3:0] Q, input wire CLK);

// Wire de salidas nQ (Q negada)
wire [3:0] nQ;

// Wires que van directos a las entradas J y K de los respectivos biestables.
wire wJ3, wK3, wJ2, wK2, wJ1, wK1, wJ0, wK0;

// Wires que conectan puertas lógicas entre sí
wire j3_t1, j3_t2;
wire k3_t1, k3_t2, k3_t3;
wire j1_t1, j1_t2;
wire k1_t1, k1_t2;
wire j0_t1, j0_t2, j0_t3;
wire k0_t1;

// Cableado de los J y los K
// J3 = nQ1Q0 + Q2Q1nQ0
and g_j3_1 (j3_t1, nQ[1], Q[0]);
and g_j3_2 (j3_t2, Q[2], Q[1], nQ[0]);
or g_J3 (wJ3, j3_t1, j3_t2);

// K3 = Q2nQ1nQ0 + Q2Q1Q0 + nQ2Q1nQ0
and g_k3_1 (k3_t1, Q[2], nQ[1], nQ[0]);
and g_k3_2 (k3_t2, Q[2], Q[1], Q[0]);
and g_k3_3 (k3_t3, nQ[2], Q[1], nQ[0]);
or g_K3 (wK3, k3_t1, k3_t2, k3_t3);

// J2 = nQ3Q1Q0
and g_J2 (wJ2, nQ[3], Q[1], Q[0]);

// K2 = Q3Q1Q0
and g_K2 (wK2, Q[3], Q[1], Q[0]);

// J1 = nQ3Q2Q0 + Q3nQ2nQ0
and g_j1_1 (j1_t1, nQ[3], Q[2], Q[0]);
and g_j1_2 (j1_t2, Q[3], nQ[2], nQ[0]);
or g_J1 (wJ1, j1_t1, j1_t2);
```

```

// K1 = Q3Q2Q0 + nQ3Q2nQ0
and g_k1_1 (k1_t1, Q[3], Q[2], Q[0]);
and g_k1_2 (k1_t2, nQ[3], Q[2], nQ[0]);
or g_K1 (wK1, k1_t1, k1_t2);

// J0 = nQ3nQ2 + Q3Q1 + Q2nQ1
and g_j0_1 (j0_t1, nQ[3], nQ[2]);
and g_j0_2 (j0_t2, Q[3], Q[1]);
and g_j0_3 (j0_t3, Q[2], nQ[1]);
or g_J0 (wJ0, j0_t1, j0_t2, j0_t3);

// K0 = nQ2 + Q3 ^ Q1
xor g_k0_1 (k0_t1, Q[3], Q[1]);
or g_K0 (wK0, nQ[2], k0_t1);

// Conectamos los wires a las entradas J y K de los biestables
JK jk3 (Q[3], nQ[3], wJ3, wK3, CLK);
JK jk2 (Q[2], nQ[2], wJ2, wK2, CLK);
JK jk1 (Q[1], nQ[1], wJ1, wK1, CLK);
JK jk0 (Q[0], nQ[0], wJ0, wK0, CLK);
endmodule

module test;
    reg C; // CLOCK
    wire [3:0] Q; // Salidas
    Contador counter (Q,C); // Instancia del contador

    // El clock oscila cada 10 unidades de tiempo.
    always begin
        #10 C=~C;
    end

    initial begin

        // Estados inválidos:
        // 0 2 4 7 9 11 13 14
        // 0000 0010 0100 0111 1001 1011 1101 1110
        // Ponemos estos números binarios en estas líneas para comprobar cómo desembocan en
        // estados válidos.

        // Ahora, empezamos por el estado 6 (0110 en binario):
        counter.jk3.Q = 'b0;
        counter.jk2.Q = 'b1;
        counter.jk1.Q = 'b1;
        counter.jk0.Q = 'b0;

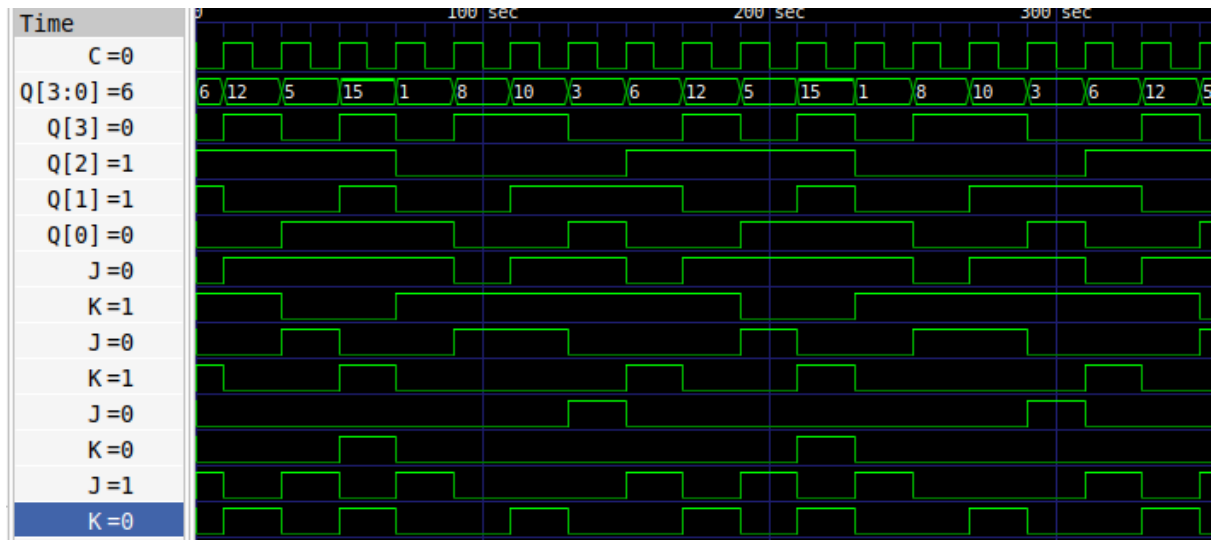
        // Archivo para GTKWave (opcional si solo quieres ver consola)
        $dumpfile("contador_arbitrario.dmp");
        $dumpvars(2, counter, Q);
        C=0;

        // Monitoreamos la señal de reloj para observar los cambios en la salida.
        // Imprimimos la salida en binario para ver el correcto funcionamiento del contador.
        $monitor($time, ": Clock: %b | Salida Q: %d (Binario: %b)", C, Q, Q);
        #1000 $finish; // terminamos el programa a las 1000 unidades de tiempo para
        // evitar entrar en un bucle infinito.
    end
endmodule

```

8. Ejemplos de pruebas (Con y sin GTKWave)

Tras ejecutar el código y abrir el archivo «contador_arbitrario.dmp» con el software GTKwave, observamos el siguiente diagrama temporal:



Como podemos contemplar, el contador comienza por el número 6, siguiendo la secuencia dada en el enunciado y reiniciándose tras pasar del estado 3. Por tanto, el objetivo principal de la práctica está cumplido.

```
● alumno@a2fc01:~/Escritorio/veriveri$ ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
0: Clock: 0 | Salida Q: 6 (Binario: 0110)
10: Clock: 1 | Salida Q: 12 (Binario: 1100)
20: Clock: 0 | Salida Q: 12 (Binario: 1100)
30: Clock: 1 | Salida Q: 5 (Binario: 0101)
40: Clock: 0 | Salida Q: 5 (Binario: 0101)
50: Clock: 1 | Salida Q: 15 (Binario: 1111)
60: Clock: 0 | Salida Q: 15 (Binario: 1111)
70: Clock: 1 | Salida Q: 1 (Binario: 0001)
80: Clock: 0 | Salida Q: 1 (Binario: 0001)
90: Clock: 1 | Salida Q: 8 (Binario: 1000)
100: Clock: 0 | Salida Q: 8 (Binario: 1000)
110: Clock: 1 | Salida Q: 10 (Binario: 1010)
120: Clock: 0 | Salida Q: 10 (Binario: 1010)
130: Clock: 1 | Salida Q: 3 (Binario: 0011)
140: Clock: 0 | Salida Q: 3 (Binario: 0011)
150: Clock: 1 | Salida Q: 6 (Binario: 0110)
160: Clock: 0 | Salida Q: 6 (Binario: 0110)
```

Sin embargo, también debemos observar el comportamiento del programa con los estados inválidos para confirmar su perfecto funcionamiento, para lo cual cambiaremos el estado inicial del contador modificando las siguientes cuatro líneas de código y sustituyendo este número por la representación en binario del estado a comprobar. Por ejemplo, si quisiéramos comprobar el estado 2 (0010 en binario), pondríamos en nuestro código:

```
counter.jk3.Q = 'b0;
counter.jk2.Q = 'b0;
counter.jk1.Q = 'b1;
counter.jk0.Q = 'b0;
```

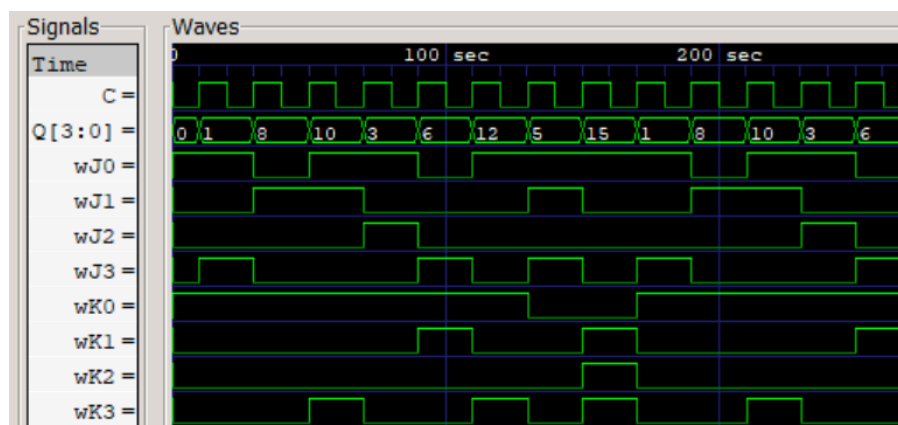
Pero como el estado inicial de nuestro contador es el 6, en nuestro archivo de código pone lo que aparece en la imagen de la derecha:

```
counter.jk3.Q = 'b0;
counter.jk2.Q = 'b1;
counter.jk1.Q = 'b1;
counter.jk0.Q = 'b0;
```

Pasemos a comprobar todos los estados inválidos: 0, 2, 4, 7, 9, 11, 13 y 14:

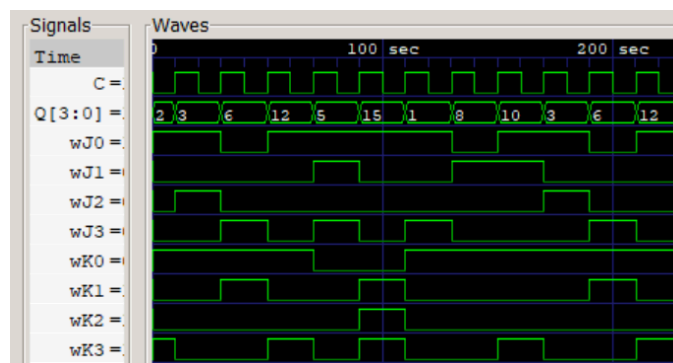
Probemos con el 0 (0000 en binario):

```
● kali@PC-de-David:/mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
 0: Clock: 0 | Salida Q: 0 (Binario: 0000)
10: Clock: 1 | Salida Q: 1 (Binario: 0001)
20: Clock: 0 | Salida Q: 1 (Binario: 0001)
30: Clock: 1 | Salida Q: 8 (Binario: 1000)
40: Clock: 0 | Salida Q: 8 (Binario: 1000)
50: Clock: 1 | Salida Q: 10 (Binario: 1010)
60: Clock: 0 | Salida Q: 10 (Binario: 1010)
70: Clock: 1 | Salida Q: 3 (Binario: 0011)
80: Clock: 0 | Salida Q: 3 (Binario: 0011)
90: Clock: 1 | Salida Q: 6 (Binario: 0110)
100: Clock: 0 | Salida Q: 6 (Binario: 0110)
```



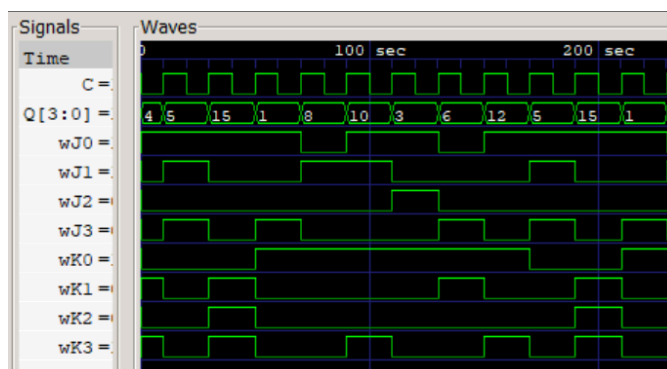
Ahora con el 2 (0010 en binario):

```
kali@PC-de-David: /mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
0: Clock: 0 | Salida Q: 2 (Binario: 0010)
10: Clock: 1 | Salida Q: 3 (Binario: 0011)
20: Clock: 0 | Salida Q: 3 (Binario: 0011)
30: Clock: 1 | Salida Q: 6 (Binario: 0110)
40: Clock: 0 | Salida Q: 6 (Binario: 0110)
50: Clock: 1 | Salida Q: 12 (Binario: 1100)
60: Clock: 0 | Salida Q: 12 (Binario: 1100)
70: Clock: 1 | Salida Q: 5 (Binario: 0101)
80: Clock: 0 | Salida Q: 5 (Binario: 0101)
90: Clock: 1 | Salida Q: 15 (Binario: 1111)
100: Clock: 0 | Salida Q: 15 (Binario: 1111)
110: Clock: 1 | Salida Q: 1 (Binario: 0001)
120: Clock: 0 | Salida Q: 1 (Binario: 0001)
130: Clock: 1 | Salida Q: 8 (Binario: 1000)
140: Clock: 0 | Salida Q: 8 (Binario: 1000)
150: Clock: 1 | Salida Q: 10 (Binario: 1010)
160: Clock: 0 | Salida Q: 10 (Binario: 1010)
170: Clock: 1 | Salida Q: 3 (Binario: 0011)
180: Clock: 0 | Salida Q: 3 (Binario: 0011)
190: Clock: 1 | Salida Q: 6 (Binario: 0110)
200: Clock: 0 | Salida Q: 6 (Binario: 0110)
```



Procedemos con el 4 (0100 en binario):

```
kali@PC-de-David: /mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
0: Clock: 0 | Salida Q: 4 (Binario: 0100)
10: Clock: 1 | Salida Q: 5 (Binario: 0101)
20: Clock: 0 | Salida Q: 5 (Binario: 0101)
30: Clock: 1 | Salida Q: 15 (Binario: 1111)
40: Clock: 0 | Salida Q: 15 (Binario: 1111)
50: Clock: 1 | Salida Q: 1 (Binario: 0001)
60: Clock: 0 | Salida Q: 1 (Binario: 0001)
70: Clock: 1 | Salida Q: 8 (Binario: 1000)
80: Clock: 0 | Salida Q: 8 (Binario: 1000)
90: Clock: 1 | Salida Q: 10 (Binario: 1010)
100: Clock: 0 | Salida Q: 10 (Binario: 1010)
110: Clock: 1 | Salida Q: 3 (Binario: 0011)
120: Clock: 0 | Salida Q: 3 (Binario: 0011)
130: Clock: 1 | Salida Q: 6 (Binario: 0110)
140: Clock: 0 | Salida Q: 6 (Binario: 0110)
```

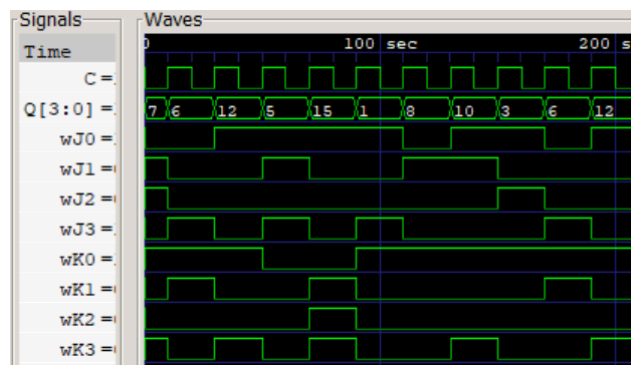


Pasamos a comprobar el 7 (0111 en binario):

```

kali@PC-de-David:~/mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
 0: Clock: 0 | Salida Q: 7 (Binario: 0111)
10: Clock: 1 | Salida Q: 6 (Binario: 0110)
20: Clock: 0 | Salida Q: 6 (Binario: 0110)
30: Clock: 1 | Salida Q: 12 (Binario: 1100)
40: Clock: 0 | Salida Q: 12 (Binario: 1100)
50: Clock: 1 | Salida Q: 5 (Binario: 0101)
60: Clock: 0 | Salida Q: 5 (Binario: 0101)
70: Clock: 1 | Salida Q: 15 (Binario: 1111)
80: Clock: 0 | Salida Q: 15 (Binario: 1111)
90: Clock: 1 | Salida Q: 1 (Binario: 0001)
100: Clock: 0 | Salida Q: 1 (Binario: 0001)
110: Clock: 1 | Salida Q: 8 (Binario: 1000)
120: Clock: 0 | Salida Q: 8 (Binario: 1000)
130: Clock: 1 | Salida Q: 10 (Binario: 1010)
140: Clock: 0 | Salida Q: 10 (Binario: 1010)
150: Clock: 1 | Salida Q: 3 (Binario: 0011)
160: Clock: 0 | Salida Q: 3 (Binario: 0011)
170: Clock: 1 | Salida Q: 6 (Binario: 0110)
180: Clock: 0 | Salida Q: 6 (Binario: 0110)

```

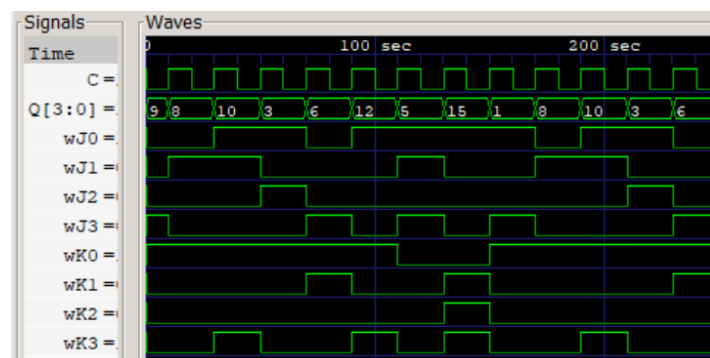


Ahora probemos con el 9 (1001 en binario):

```

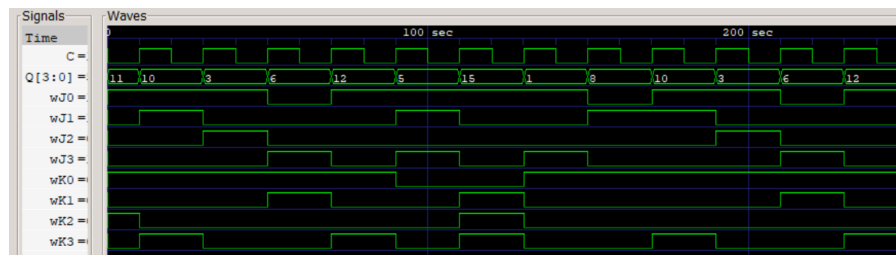
kali@PC-de-David:~/mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
 0: Clock: 0 | Salida Q: 9 (Binario: 1001)
10: Clock: 1 | Salida Q: 8 (Binario: 1000)
20: Clock: 0 | Salida Q: 8 (Binario: 1000)
30: Clock: 1 | Salida Q: 10 (Binario: 1010)
40: Clock: 0 | Salida Q: 10 (Binario: 1010)
50: Clock: 1 | Salida Q: 3 (Binario: 0011)
60: Clock: 0 | Salida Q: 3 (Binario: 0011)
70: Clock: 1 | Salida Q: 6 (Binario: 0110)
80: Clock: 0 | Salida Q: 6 (Binario: 0110)
90: Clock: 1 | Salida Q: 12 (Binario: 1100)
100: Clock: 0 | Salida Q: 12 (Binario: 1100)
110: Clock: 1 | Salida Q: 5 (Binario: 0101)
120: Clock: 0 | Salida Q: 5 (Binario: 0101)
130: Clock: 1 | Salida Q: 15 (Binario: 1111)
140: Clock: 0 | Salida Q: 15 (Binario: 1111)
150: Clock: 1 | Salida Q: 1 (Binario: 0001)
160: Clock: 0 | Salida Q: 1 (Binario: 0001)
170: Clock: 1 | Salida Q: 8 (Binario: 1000)
180: Clock: 0 | Salida Q: 8 (Binario: 1000)

```



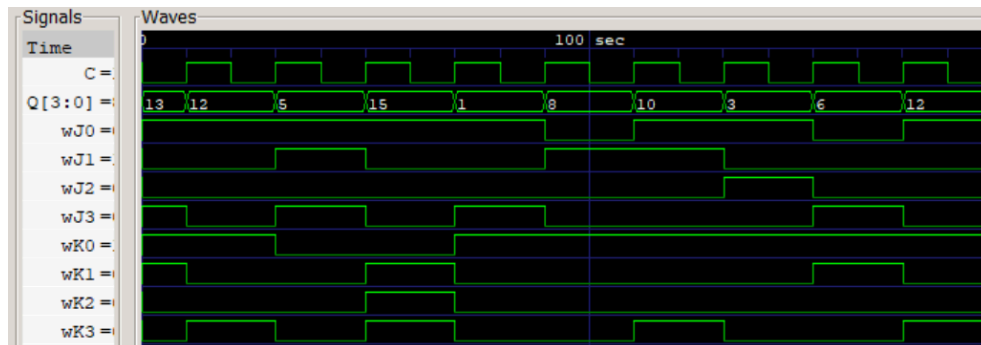
Continuemos con el 11 (1011 en binario):

```
● kali@PC-de-David: /mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
  0: Clock: 0 | Salida Q: 11 (Binario: 1011)
 10: Clock: 1 | Salida Q: 10 (Binario: 1010)
 20: Clock: 0 | Salida Q: 10 (Binario: 1010)
 30: Clock: 1 | Salida Q: 3 (Binario: 0011)
 40: Clock: 0 | Salida Q: 3 (Binario: 0011)
 50: Clock: 1 | Salida Q: 6 (Binario: 0110)
 60: Clock: 0 | Salida Q: 6 (Binario: 0110)
 70: Clock: 1 | Salida Q: 12 (Binario: 1100)
 80: Clock: 0 | Salida Q: 12 (Binario: 1100)
 90: Clock: 1 | Salida Q: 5 (Binario: 0101)
100: Clock: 0 | Salida Q: 5 (Binario: 0101)
110: Clock: 1 | Salida Q: 15 (Binario: 1111)
120: Clock: 0 | Salida Q: 15 (Binario: 1111)
130: Clock: 1 | Salida Q: 1 (Binario: 0001)
140: Clock: 0 | Salida Q: 1 (Binario: 0001)
150: Clock: 1 | Salida Q: 8 (Binario: 1000)
160: Clock: 0 | Salida Q: 8 (Binario: 1000)
170: Clock: 1 | Salida Q: 10 (Binario: 1010)
180: Clock: 0 | Salida Q: 10 (Binario: 1010)
```



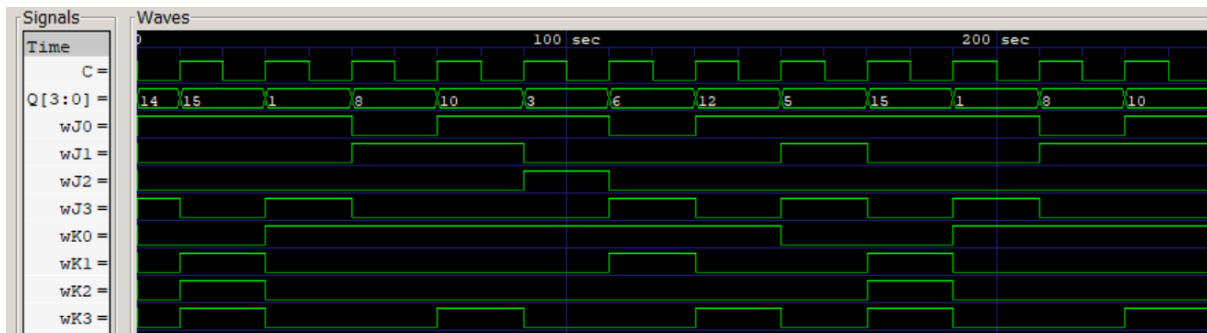
Procedamos con el 13 (1101 en binario):

```
● kali@PC-de-David: /mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
  0: Clock: 0 | Salida Q: 13 (Binario: 1101)
 10: Clock: 1 | Salida Q: 12 (Binario: 1100)
 20: Clock: 0 | Salida Q: 12 (Binario: 1100)
 30: Clock: 1 | Salida Q: 5 (Binario: 0101)
 40: Clock: 0 | Salida Q: 5 (Binario: 0101)
 50: Clock: 1 | Salida Q: 15 (Binario: 1111)
 60: Clock: 0 | Salida Q: 15 (Binario: 1111)
 70: Clock: 1 | Salida Q: 1 (Binario: 0001)
 80: Clock: 0 | Salida Q: 1 (Binario: 0001)
 90: Clock: 1 | Salida Q: 8 (Binario: 1000)
100: Clock: 0 | Salida Q: 8 (Binario: 1000)
110: Clock: 1 | Salida Q: 10 (Binario: 1010)
120: Clock: 0 | Salida Q: 10 (Binario: 1010)
130: Clock: 1 | Salida Q: 3 (Binario: 0011)
140: Clock: 0 | Salida Q: 3 (Binario: 0011)
150: Clock: 1 | Salida Q: 6 (Binario: 0110)
160: Clock: 0 | Salida Q: 6 (Binario: 0110)
170: Clock: 1 | Salida Q: 12 (Binario: 1100)
180: Clock: 0 | Salida Q: 12 (Binario: 1100)
```



Y, finalmente, con el 14:

```
● kali@PC-de-David:/mnt/c/Users/david/Desktop/PROGRAMACION/COMPUTADORES_VERILOG$ iverilog main.v -o main | ./main
VCD info: dumpfile contador_arbitrario.dmp opened for output.
  0: Clock: 0 | Salida Q: 14 (Binario: 1110)
 10: Clock: 1 | Salida Q: 15 (Binario: 1111)
 20: Clock: 0 | Salida Q: 15 (Binario: 1111)
 30: Clock: 1 | Salida Q: 1 (Binario: 0001)
 40: Clock: 0 | Salida Q: 1 (Binario: 0001)
 50: Clock: 1 | Salida Q: 8 (Binario: 1000)
 60: Clock: 0 | Salida Q: 8 (Binario: 1000)
 70: Clock: 1 | Salida Q: 10 (Binario: 1010)
 80: Clock: 0 | Salida Q: 10 (Binario: 1010)
 90: Clock: 1 | Salida Q: 3 (Binario: 0011)
100: Clock: 0 | Salida Q: 3 (Binario: 0011)
110: Clock: 1 | Salida Q: 6 (Binario: 0110)
120: Clock: 0 | Salida Q: 6 (Binario: 0110)
130: Clock: 1 | Salida Q: 12 (Binario: 1100)
140: Clock: 0 | Salida Q: 12 (Binario: 1100)
150: Clock: 1 | Salida Q: 5 (Binario: 0101)
160: Clock: 0 | Salida Q: 5 (Binario: 0101)
170: Clock: 1 | Salida Q: 15 (Binario: 1111)
180: Clock: 0 | Salida Q: 15 (Binario: 1111)
```



Con esto confirmamos el correcto funcionamiento del contador y, por tanto, la completitud del objetivo principal de esta práctica.

9. Conclusiones, referencias y bibliografía

Conclusiones

La realización de esta práctica, en nuestra opinión, ha supuesto una absoluta adquisición de las competencias incluidas en la guía docente y referidas a la práctica, sobre todo la CC9:

- «CC9.- *Capacidad de conocer, comprender y evaluar la estructura y arquitectura de los computadores, así como los componentes básicos que los conforman*».

Añadidamente, creemos que esta práctica ha supuesto la mejora en nuestras habilidades técnicas de: programación, circuitería, análisis de tablas y mapas de datos, ofimática, manejo de otros sistemas operativos (como Linux), manejo de la terminal de comandos del propio sistema... Y hemos observado también una mejora en las habilidades blandas de: trabajo en equipo, detección de fallos, resolución eficiente de problemas y comunicación y expresión tanto orales (la defensa) como escritas (este propio informe).

Referencias

[1] : Minecraft Forum (<https://www.minecraftforum.net/forums/minecraft-java-edition/redstone-discussion-and/345962-so-anyone-have-a-compact-jk-flipflop-yet>)

Bibliografía

BRAVO PLASENCIA, R. (2024). *Circuitos secuenciales*.

GONZÁLEZ TALAVÁN, G. GGTSOFT, DIA USAL. (2025, 2010-2019). *Página oficial de Computadores I. Prácticas de la 1 a la 8*. avellano.usal.es/compi.

HAYES, J.P. (1996). *Introducción al diseño lógico digital*. Editorial Addison-Wesley.

HERNÁNDEZ DE LA IGLESIA, D., DIA USAL. (2025). *Computadores I. 1º Grado en Ingeniería Informática*. (Apuntes de la asignatura).

LITTLE S. et al. (30 mayo 2014). *Verilog-AMS Language Reference Manual*. Accellera Systems Initiative.